

**Note**[↑ Back to top](#)

This tutorial is compatible with Gymnasium version 1.2.3.

Make your own custom environment

This tutorial shows how to create new environment and links to relevant useful wrappers, utilities and tests included in Gymnasium.

Setup

Recommended solution

1. Install `pipx` following the [pipx documentation](#).
2. Then install Copier:

```
pipx install copier
```

Alternative solutions

Install Copier with Pip or Conda:

```
pip install copier
```

[↑ Back to top](#)

Generate your environment

You can check that `copier` has been correctly installed by running the following command, which should output a version number:

```
copier --version
```

Then you can just run the following command and replace the string `path/to/directory` by the path to the directory where you want to create your new project.

```
copier copy https://github.com/Farama-Foundation/gymnasium-env-template.git "path/to/directory"
```

Answer the questions, and when it's finished you should get a project structure like the following:

```
.
├── gymnasium_env
│   ├── envs
│   │   ├── grid_world.py
│   │   └── __init__.py
│   ├── __init__.py
│   └── wrappers
│       ├── clip_reward.py
│       ├── discrete_actions.py
│       ├── __init__.py
│       ├── reacher_weighted_reward.py
│       └── relative_position.py
```



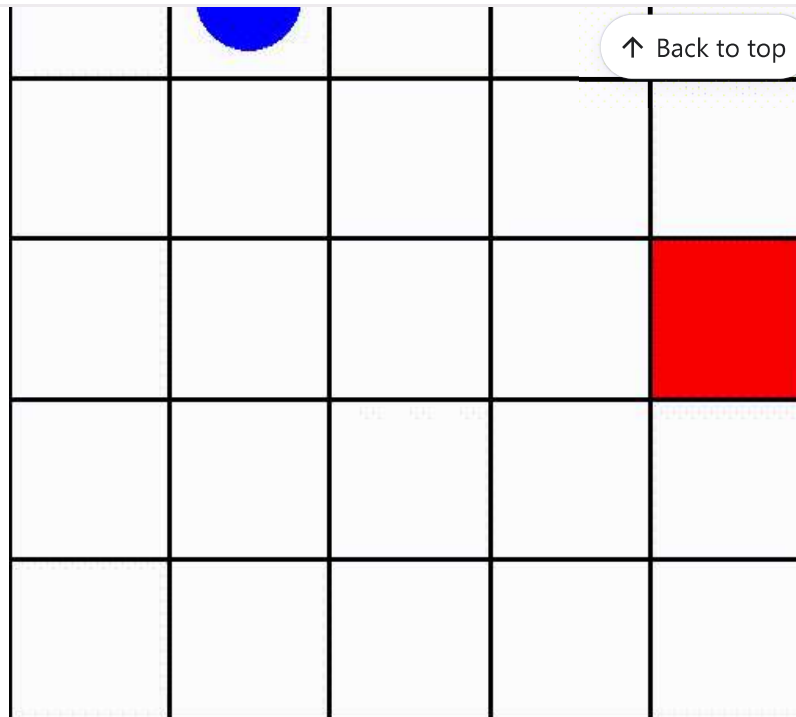
Before learning how to create your own environment, you should check out [the documentation of Gymnasium's API](#).

↑ Back to top

To illustrate the process of subclassing `gymnasium.Env`, we will implement a very simplistic game, called `GridWorldEnv`. We will write the code for our custom environment in `gymnasium_env/envs/grid_world.py`. The environment consists of a 2-dimensional square grid of fixed size (specified via the `size` parameter during construction). The agent can move vertically or horizontally between grid cells in each timestep. The goal of the agent is to navigate to a target on the grid that has been placed randomly at the beginning of the episode.

- Observations provide the location of the target and agent.
- There are 4 actions in our environment, corresponding to the movements "right", "up", "left", and "down".
- A done signal is issued as soon as the agent has navigated to the grid cell where the target is located.
- Rewards are binary and sparse, meaning that the immediate reward is always zero, unless the agent has reached the target, then it is 1.

An episode in this environment (with `size=5`) might look like this:



where the blue dot is the agent and the red square represents the target.

Let us look at the source code of `GridWorldEnv` piece by piece:

Declaration and Initialization

Our custom environment will inherit from the abstract class `gymnasium.Env`. You shouldn't forget to add the `metadata` attribute to your class. There, you should specify the render-modes that are supported by your environment (e.g., `"human"`, `"rgb_array"`, `"ansi"`) and the framerate at which your environment should be rendered. Every environment should support `None` as render-mode; you don't need to add it



`self.action_space`. In our case, observations should contain information about the location of the agent and target on the 2-dimensional grid. We will choose to represent observations in the form of dictionaries with keys `"agent"` and `"target"`. An observation may look like `{"agent": array([1, 0]), "target": array([0, 3])}`. Since we have 4 actions in our environment ("right", "up", "left", "down"), we will use `Discrete(4)` as an action space. Here is the declaration of `GridWorldEnv` and the implementation of `__init__`:

[↑ Back to top](#)

[↑ Back to top](#)

```
import numpy as np
import pygame

import gymnasium as gym
from gymnasium import spaces

class Actions(Enum):
    RIGHT = 0
    UP = 1
    LEFT = 2
    DOWN = 3

class GridWorldEnv(gym.Env):
    metadata = {"render_modes": ["human", "rgb_array"], "render_fps": 4}

    def __init__(self, render_mode=None, size=5):
        self.size = size # The size of the square grid
        self.window_size = 512 # The size of the PyGame window

        # Observations are dictionaries with the agent's and the target's location.
        # Each location is encoded as an element of {0, ..., `size`}^2, i.e. MultiDiscrete([size, size]).
        self.observation_space = spaces.Dict(
            {
                "agent": spaces.Box(0, size - 1, shape=(2,), dtype=int),
                "target": spaces.Box(0, size - 1, shape=(2,), dtype=int),
            }
        )
        self.agent_location = np.array([-1, -1], dtype=int)
```



The following dictionary maps abstract actions from ``self.action_space`` to the direction we will walk in if that action is taken.

i.e. 0 corresponds to "right", 1 to "up" etc.

Uses NumPy [row, col] convention where row 0 is at the top.

"""

```
self._action_to_direction = {
    Actions.RIGHT.value: np.array([0, 1]),
    Actions.UP.value: np.array([-1, 0]),
    Actions.LEFT.value: np.array([0, -1]),
    Actions.DOWN.value: np.array([1, 0]),
}
```

```
assert render_mode is None or render_mode in self.metadata["render_modes"]
self.render_mode = render_mode
```

"""

If human-rendering is used, ``self.window`` will be a reference to the window that we draw to. ``self.clock`` will be a clock that is used to ensure that the environment is rendered at the correct framerate in human-mode. They will remain ``None`` until human-mode is used for the first time.

"""

```
self.window = None
self.clock = None
```

[↑ Back to top](#)

Constructing Observations From Environment States

Since we will need to compute observations both in `reset` and `step`, it is often convenient to have a

`compute_observation` method that computes the observation from the current state.

[↑ Back to top](#)

We can also implement a similar method for the auxiliary information that is returned by `step` and `reset`. In our case, we would like to provide the manhattan distance between the agent and the target:

```
def _get_info(self):  
    return {  
        "distance": np.linalg.norm(  
            self._agent_location - self._target_location, ord=1  
        )  
    }
```

Oftentimes, info will also contain some data that is only available inside the `step` method (e.g., individual reward terms). In that case, we would have to update the dictionary that is returned by `_get_info` in `step`.

Reset

The `reset` method will be called to initiate a new episode. You may assume that the `step` method will not be called before `reset` has been called. Moreover, `reset` should be called whenever a done signal has been issued. Users may pass the `seed` keyword to `reset` to initialize any random number generator that is used by the environment to a deterministic state. It is recommended to use the random number generator `self.np_random` that is provided by the environment's base class, `gymnasium.Env`. If you only use this RNG, you do not need to worry much about seeding, *but you need to remember to call* ```super().reset(seed=seed)``` to make sure that `gymnasium.Env` correctly seeds the RNG. Once this is done, we can randomly set the state of our environment. In our case, we randomly choose the agent's location and the random sample target positions, until it does not coincide with the agent's position.



```
super().reset(seed=seed)
```

[↑ Back to top](#)

```
# Choose the agent's location uniformly at random
```

```
self._agent_location = self.np_random.integers(0, self.size, size=2, dtype=int)
```

```
# We will sample the target's location randomly until it does not coincide with the agent's location
```

```
self._target_location = self._agent_location
```

```
while np.array_equal(self._target_location, self._agent_location):
```

```
    self._target_location = self.np_random.integers(
```

```
        0, self.size, size=2, dtype=int
```

```
    )
```

```
observation = self._get_obs()
```

```
info = self._get_info()
```

```
if self.render_mode == "human":
```

```
    self._render_frame()
```

```
return observation, info
```

Step

The `step` method usually contains most of the logic of your environment. It accepts an `action`, computes the state of the environment after applying that action and returns the 5-tuple `(observation, reward, terminated, truncated, info)`. See [gymnasium.Env.step\(\)](#). Once the new state of the environment has been computed, we can check whether it is a terminal state and we set `done` accordingly. Since we are using sparse binary rewards in `GridWorldEnv`, computing `reward` is trivial once we know `done`. To gather `observation` and `info`, we can again make use of `_get_obs` and `_get_info`:



```
direction = self._action_to_direction[action]
# We use `np.clip` to make sure we don't go out of bounds
self._agent_location = np.clip(
    self._agent_location + direction, 0, self.size - 1
)
# An episode is done iff the agent has reached the target
terminated = np.array_equal(self._agent_location, self._target_location)
reward = 1 if terminated else 0 # Binary sparse rewards
observation = self._get_obs()
info = self._get_info()

if self.render_mode == "human":
    self._render_frame()

return observation, reward, terminated, False, info
```

[↑ Back to top](#)

Rendering

Here, we are using PyGame for rendering. A similar approach to rendering is used in many environments that are included with Gymnasium and you can use it as a skeleton for your own environments:



```
return self._render_frame()
```

[↑ Back to top](#)

```
def _render_frame(self):
    if self.window is None and self.render_mode == "human":
        pygame.init()
        pygame.display.init()
        self.window = pygame.display.set_mode(
            (self.window_size, self.window_size)
        )
    if self.clock is None and self.render_mode == "human":
        self.clock = pygame.time.Clock()

    canvas = pygame.Surface((self.window_size, self.window_size))
    canvas.fill((255, 255, 255))
    pix_square_size = (
        self.window_size / self.size
    ) # The size of a single grid square in pixels

    # First we draw the target
    # Convert [row, col] to pygame (x, y) by reversing the coordinates
    pygame.draw.rect(
        canvas,
        (255, 0, 0),
        pygame.Rect(
            pix_square_size * self._target_location[::-1],
            (pix_square_size, pix_square_size),
        ),
    )
    # Now we draw the agent
    pygame.draw.circle(
        canvas,
```



```
for x in range(self.size + 1):
    pygame.draw.line(
        canvas,
        0,
        (0, pix_square_size * x),
        (self.window_size, pix_square_size * x),
        width=3,
    )
    pygame.draw.line(
        canvas,
        0,
        (pix_square_size * x, 0),
        (pix_square_size * x, self.window_size),
        width=3,
    )

if self.render_mode == "human":
    # The following line copies our drawings from `canvas` to the visible window
    self.window.blit(canvas, canvas.get_rect())
    pygame.event.pump()
    pygame.display.update()

    # We need to ensure that human-rendering occurs at the predefined framerate.
    # The following line will automatically add a delay to keep the framerate stable.
    self.clock.tick(self.metadata["render_fps"])
else: # rgb_array
    return np.transpose(
        np.array(pygame.surfarray.pixels3d(canvas)), axes=(1, 0, 2)
    )
```

[↑ Back to top](#)



The `close` method should close any open resources that were used by the environment. In many cases, you don't actually have to bother to implement it. However, in our example `render_mode` may be `"human"` and we might need to close the window that has been opened:

[↑ Back to top](#)

```
def close(self):
    if self.window is not None:
        pygame.display.quit()
        pygame.quit()
```

In other environments `close` might also close files that were opened or release other resources. You shouldn't interact with the environment after having called `close`.

Registering Envs

In order for the custom environments to be detected by Gymnasium, they must be registered as follows. We will choose to put this code in `gymnasium_env/__init__.py`.

```
from gymnasium.envs.registration import register

register(
    id="gymnasium_env/GridWorld-v0",
    entry_point="gymnasium_env.envs:GridWorldEnv",
)
```

The environment ID consists of three components, two of which are optional: an optional namespace (here: `gymnasium_env`), a mandatory name (here: `GridWorld`) and an optional but recommended version



[documentation](#) for more information). A done signal can be produced if the agent has reached the target or 300 steps have been executed in the current episode. To distinguish truncation and termination, you can check `info["TimeLimit.truncated"]`.

[↑ Back to top](#)

Apart from `id` and `entrypoint`, you may pass the following additional keyword arguments to `register`:

Name	Type	Default	Description
<code>reward_threshold</code>	<code>float</code>	<code>None</code>	The reward threshold before the task is considered solved
<code>nondeterministic</code>	<code>bool</code>	<code>False</code>	Whether this environment is non-deterministic even after seeding
<code>max_episode_steps</code>	<code>int</code>	<code>None</code>	The maximum number of steps that an episode can consist of. If not <code>None</code> , a <code>TimeLimit</code> wrapper is added
<code>order_enforce</code>	<code>bool</code>	<code>True</code>	Whether to wrap the environment in an <code>OrderEnforcing</code> wrapper
<code>kwargs</code>	<code>dict</code>	<code>{}</code>	The default kwargs to pass to the environment class

Most of these keywords (except for `max_episode_steps`, `order_enforce` and `kwargs`) do not alter the behavior of environment instances but merely provide some extra information about your environment. After registration, our custom `GridWorldEnv` environment can be created with `env = gymnasium.make('gymnasium_env/GridWorld-v0')`.

`gymnasium_env/envs/__init__.py` should have:

```
from gymnasium_env.envs.grid_world import GridWorldEnv
```



The last step is to structure our code as a Python package. This involves configuring `pyproject.toml`. A minimal example of how to do so is as follows:

This page uses [Google Analytics](#) to collect statistics.



```
# run_gymnasium_env.py
```

[↑ Back to top](#)

```
import gymnasium
import gymnasium_env
env = gymnasium.make('gymnasium_env/GridWorld-v0')
```

You can also pass keyword arguments of your environment's constructor to `gymnasium.make` to customize the environment. In our case, we could do:

```
env = gymnasium.make('gymnasium_env/GridWorld-v0', size=10)
```

Sometimes, you may find it more convenient to skip registration and call the environment's constructor yourself. Some may find this approach more pythonic and environments that are instantiated like this are also perfectly fine (but remember to add wrappers as well!).

Using Wrappers

Oftentimes, we want to use different variants of a custom environment, or we want to modify the behavior of an environment that is provided by Gymnasium or some other party. Wrappers allow us to do this without changing the environment implementation or adding any boilerplate code. Check out the [wrapper documentation](#) for details on how to use wrappers and instructions for implementing your own. In our example, observations cannot be used directly in learning code because they are dictionaries. However, we don't actually need to touch our environment implementation to fix this! We can simply add a wrapper on top of environment instances to flatten observations into a single array:



```
from gymnasium.wrappers import FlattenObservation

env = gymnasium.make('gymnasium_env/GridWorld-v0')
wrapped_env = FlattenObservation(env)
print(wrapped_env.reset())    # E.g. [3 0 3 3], {}
```

[↑ Back to top](#)

Wrappers have the big advantage that they make environments highly modular. For instance, instead of flattening the observations from GridWorld, you might only want to look at the relative position of the target and the agent. In the section on [ObservationWrappers](#) we have implemented a wrapper that does this job. This wrapper is also available in `gymnasium_env/wrappers/relative_position.py`:

```
import gymnasium
import gymnasium_env
from gymnasium_env.wrappers import RelativePosition

env = gymnasium.make('gymnasium_env/GridWorld-v0')
wrapped_env = RelativePosition(env)
print(wrapped_env.reset())    # E.g. [-3 3], {}
```

Download Python source code:
`environment_creation.py`

Download Jupyter notebook:
`environment_creation.ipynb`



↑ Back to top